



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE CIENCIAS

**Reporte proyecto 1 (Implementación de protocolo
de chat utilizando sockets TCP).**

PRESENTA

Bruno Bernardo Soto Lugo
426008646

PROFESOR

Canek Peláez Valdés

AYUDANTES

Miriam Torres Bucio
Cynthia Lizbeth Sánchez Urbano

Índice general

1. Introducción	3
2. Análisis	4
2.1. Requisitos	4
3. Diseño	6
3.1. Diagramas UML	7
3.1.1. Diagrama de secuencia (Proceso de autenticación)	7
3.1.2. Diagrama de secuencia (Crear sala)	7
3.1.3. Diagrama de secuencia (Invitar y unirse a sala)	8
3.1.4. Diagrama de secuencia (Texto a sala)	8
3.1.5. Diagrama de secuencia (Desconexión)	9
4. Desarrollo	10

Capítulo 1

Introducción

El presente documento detalla el proceso de implementación de un protocolo que especifica una forma de comunicación chat. El sistema está compuesto por dos componentes principales: un cliente con interfaz gráfica desarrollado en **Kotlin** con el framework **Compose Multiplatform** y un servidor desarrollado en lenguaje **C** con conexión red usando sockets TCP/IP.

El objetivo principal del proyecto es implementar un protocolo de comunicación basado en objetos JSON que permita la identificación de usuarios, cambio de estados (Activo, Ausente, Ocupado), envío de mensajes privados y públicos, así como la gestión de salas de chat dinámicas. El desarrollo siguió el modelo de cascada.

Las etapas fueron:

1. **Análisis:** Definición de requisitos funcionales basados en las acciones del usuario.
2. **Diseño:** Arquitectura del sistema, diseño de la base de datos en memoria y diagramas UML.
3. **Desarrollo:** Codificación del servidor en C y la interfaz en Kotlin.
4. **Pruebas:** Validación de cada comando del protocolo contra el manual de especificaciones.

Capítulo 2

Análisis

En esta fase se definieron las capacidades que la implementación del sistema debe ofrecer según el protocolo.

2.1. Requisitos

A partir del análisis del protocolo, se identificaron los siguientes componentes:

- **Gestión de Identidad (IDENTIFY):** El sistema debe permitir al usuario unirse con un nombre único (con máximo 8 caracteres). El servidor debe validar la existencia previa y notificar a otros usuarios la entrada de nuevos miembros.
- **Gestión de Estado (STATUS):** Los usuarios pueden alternar entre **ACTIVE**, **AWAY** y **BUSY**. Cada cambio debe disparar un evento **NEW_STATUS** para los demás clientes conectados.
- **Comunicación (TEXT, PUBLIC_TEXT y ROOM_TEXT):**
 - Mensajes privados que solo el destinatario recibe.
 - Mensajes públicos dirigidos a todos los usuarios en la conexión global.
 - Mensajes dirigidos a todos los usuarios en una sala.
- **Gestión de Salas (ROOMS):** El sistema debe permitir crear salas (con máximo 16 caracteres), invitar usuarios y gestionar pertenencia a sala. Se requiere validación de que el usuario esté invitado antes de unirse.
- **Obtención de usuarios (USERS y ROOM_USERS):** El usuario puede obtener la lista de usuarios que están conectados globalmente o en una sala a la que pertenece.
- **Desconexión (DISCONNECT):** Manejo de limpieza de recursos cuando un cliente pierde conexión o decide salir, notificando su salida a las salas que pertenece y globalmente.

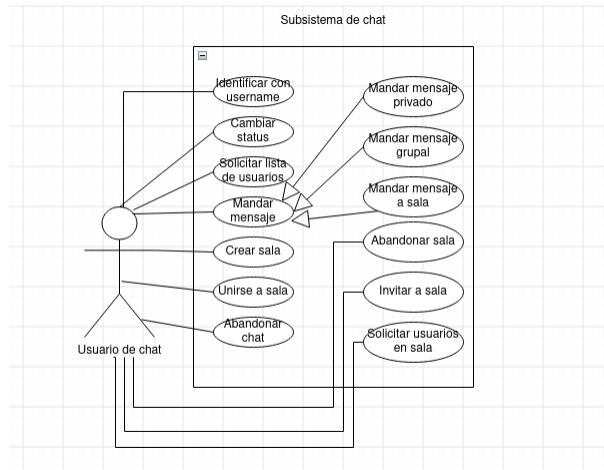


Figura 2.1: Diagrama del análisis (Use Case).

Capítulo 3

Diseño

El diseño implementa una arquitectura dividida en cliente y servidor con el objetivo de desacoplar las partes del programa y permitir que diferentes clientes puedan conectarse a un mismo servidor. El servidor actúa como un gestor de múltiples conexiones simultáneas. El cliente se encarga de la representación visual y la administración de la lógica local.

A grandes rasgos, antes de definir cualquier proceso, se define lo siguiente:

1. Una lista de usuarios global.
2. Una lista de salas global.
3. Algún tipo de estructura que guarde usuarios.
4. Algún tipo de estructura que guarde salas, una lista de miembros y una lista de usuarios invitados.

Se definen vagamente debido a que, por ahora, solo son recursos abstractos que nos ayudarán a definir los algoritmos que necesita el protocolo. En la sección de diseño se definirán concretamente.

Algunos procesos que se detallan en el protocolo son simples, y a continuación se detalla el proceso:

1. **PUBLIC_TEXT:** El cliente manda la petición. Se itera sobre una lista de usuarios global y a cada usuario se le manda el mensaje.
2. **TEXT:** El cliente manda la petición. Se busca el usuario dentro de una lista de usuarios global. Si el usuario existe, se manda el mensaje. Se regresa un error en otro caso.
3. **USERS:** El cliente manda la petición. Se itera sobre una lista de usuarios global y se agrega cada usuario con su estado a un objeto JSON. Después, este objeto JSON se incluye al objeto JSON de respuesta y se regresa esta respuesta.
4. **ROOM_USERS:** El cliente manda la petición. Se verifica pertenencia a la sala especificada y se realiza la misma acción que en **USERS**, pero utilizando la lista de usuarios de la sala y no la global.

5. **LEAVE_ROOM:** El cliente manda la petición. Se verifica pertenencia y existencia de la sala. Si alguna de las dos condiciones es falsa, se regresa un error. En otro caso, se elimina al usuario de una sala de invitados y una sala de miembros asociadas a esa sala.

Para los procesos complejos, se describirán a continuación utilizando diagramas UML.

3.1. Diagramas UML

Se detallarán en diagramas UML la lógica usada para implementar el protocolo.

3.1.1. Diagrama de secuencia (Proceso de autenticación)

El siguiente diagrama representa la secuencia que sigue el cliente y el servidor cuando un usuario busca autenticarse.

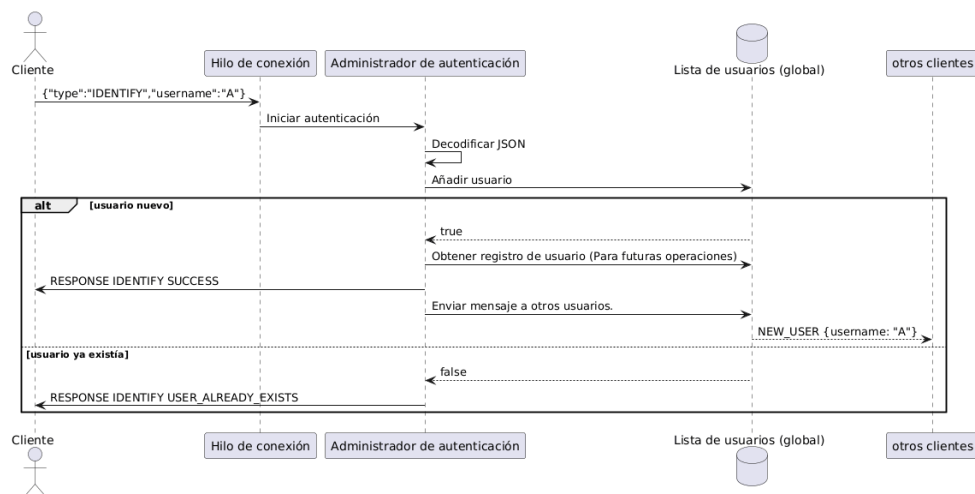


Figura 3.1: Diagrama de la autenticación.

3.1.2. Diagrama de secuencia (Crear sala)

El siguiente diagrama representa la secuencia que sigue el cliente y el servidor cuando un usuario crea una nueva sala.

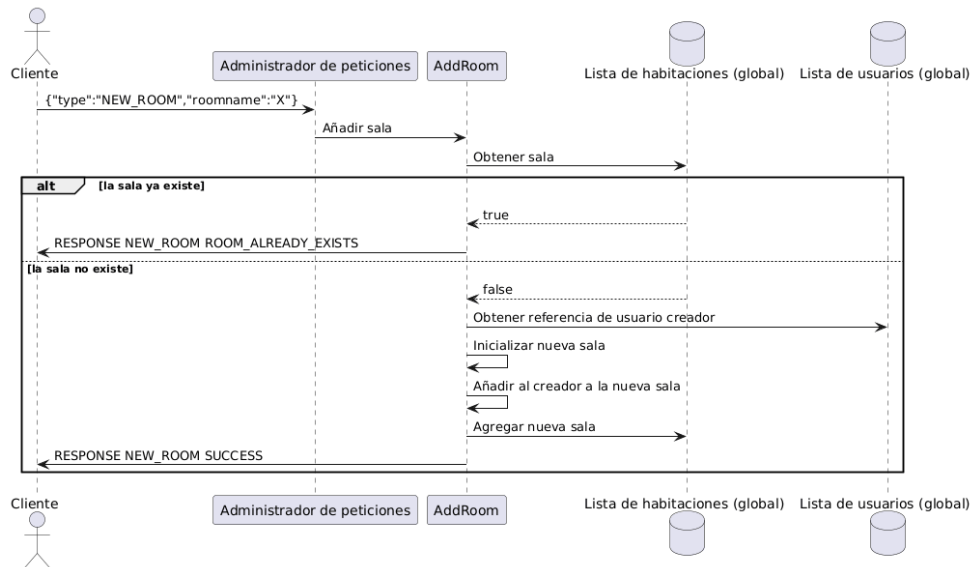


Figura 3.2: Diagrama de secuencia sobre la creación de una nueva habitación.

3.1.3. Diagrama de secuencia (Invitar y unirse a sala)

El siguiente diagrama representa la secuencia que siguen dos clientes y el servidor en el proceso de invitar y unirse a una sala.

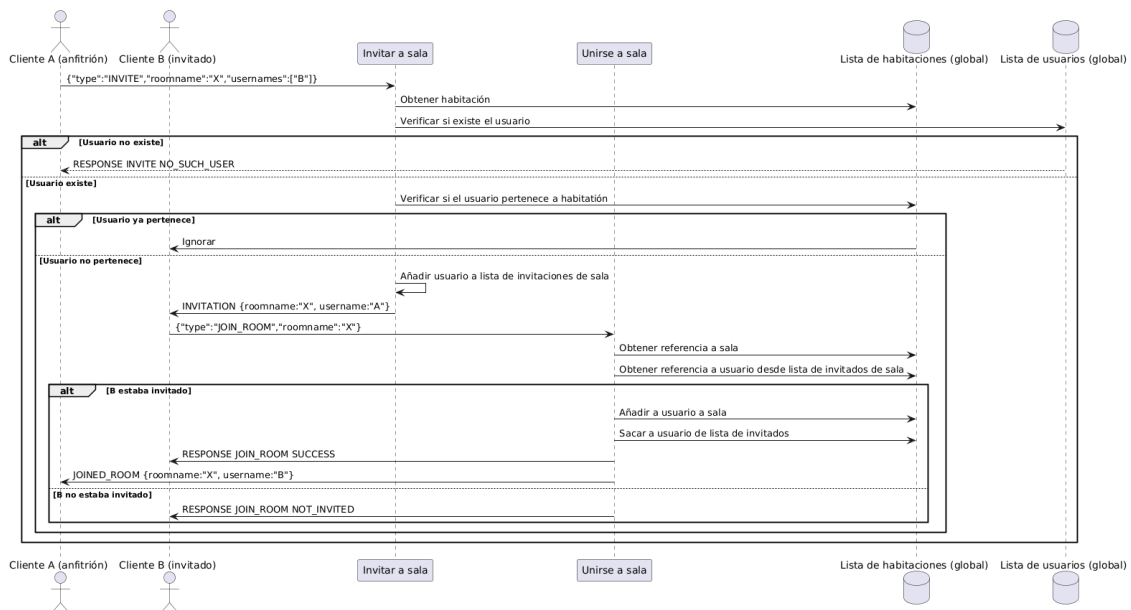


Figura 3.3: Diagrama de secuencia sobre el proceso de invitar y unirse a una sala.

3.1.4. Diagrama de secuencia (Texto a sala)

El siguiente diagrama representa la secuencia que siguen el cliente y el servidor al mandar un mensaje a una sala.

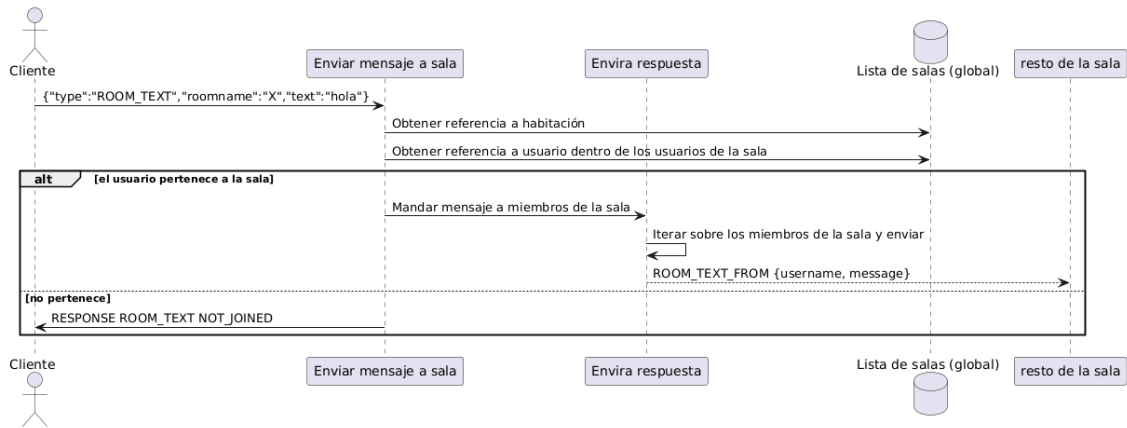


Figura 3.4: Diagrama de secuencia sobre el proceso de enviar un mensaje a una sala.

3.1.5. Diagrama de secuencia (Desconexión)

El siguiente diagrama representa la secuencia que siguen el cliente y el servidor al momento de desconectar al cliente.

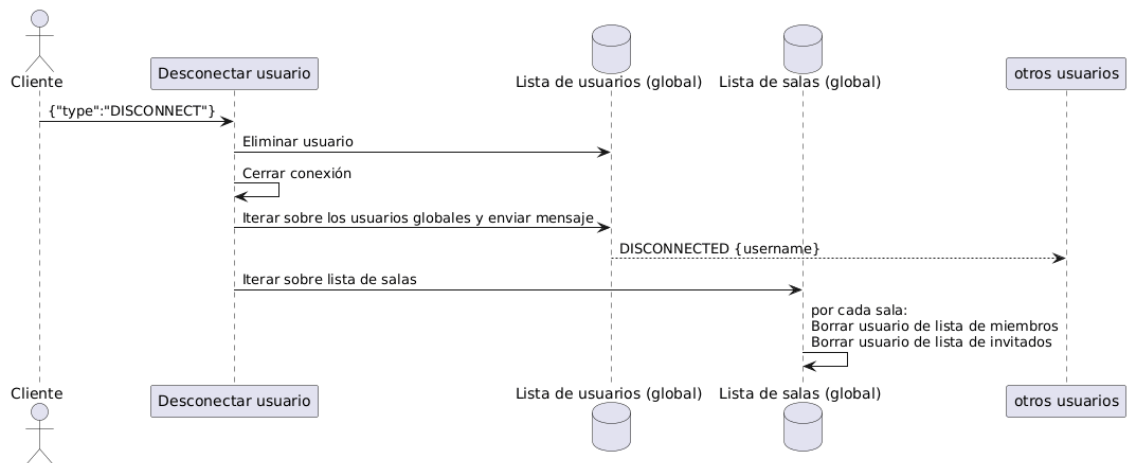


Figura 3.5: Diagrama de secuencia sobre el proceso de desconexión.

Capítulo 4

Desarrollo